

1. OJ 评测结果

```
● > ssh -p 2222 h3240105155@10.130.142.26 show 1778938726537009003
h3240105155@10.130.142.26's password:
Submission Detail:
  ID:      1778938726537009003
  Problem: nbody
  Status:  completed
  Time:    2026-05-16 13:38:46.537

Score 100.00 max.100
Judgement Message:
  Performance: 0.46ms -> 100.00/100: 5/5 correct, avg latency 0.4559ms

Judge Output:
  --- workflow 1 ---
  [step 1] $ python3 /problem/judge.py
  [judge] Done. 5/5 traces.

Traces:
  nbody-w1          PASSED
    max_atol=0.000000e+00  max_rtol=0.000000e+00  nan=false  inf=false
    latency=0.2067ms
  nbody-w2          PASSED
    max_atol=0.000000e+00  max_rtol=0.000000e+00  nan=false  inf=false
    latency=0.2752ms
  nbody-w3          PASSED
    max_atol=0.000000e+00  max_rtol=0.000000e+00  nan=false  inf=false
    latency=0.3668ms
  nbody-w4          PASSED
    max_atol=0.000000e+00  max_rtol=0.000000e+00  nan=false  inf=false
    latency=0.5449ms
  nbody-w5          PASSED
    max_atol=0.000000e+00  max_rtol=0.000000e+00  nan=false  inf=false
    latency=0.8857ms
```

```
● > ssh -p 2222 h3240105155@10.130.142.26 submit rmsnorm
h3240105155@10.130.142.26's password:
2026-05-16 13:58:19 UTC
Submitting rmsnorm
Submission ID: 1778939899949686963
2026-05-16 13:58:19.964 Waiting for GPU... (available: 4 / 4)
2026-05-16 13:58:19.964 Assigned GPU 2
2026-05-16 13:58:19.976 Submitting files
  * kernel.py : 9fb1aa67e56ce36900ecb50f23dc45c8
2026-05-16 13:58:19.996 Running Judge workflows
2026-05-16 13:58:20.021 running workflow 1 / 1
rmsnorm-w1          PASSED  atol=0.00e+00  rtol=0.00e+00  0.0582ms
rmsnorm-w2          PASSED  atol=0.00e+00  rtol=0.00e+00  0.0587ms
rmsnorm-w3          PASSED  atol=0.00e+00  rtol=0.00e+00  0.0587ms
rmsnorm-w4          PASSED  atol=0.00e+00  rtol=0.00e+00  0.0583ms
rmsnorm-w5          PASSED  atol=3.91e-03  rtol=6.49e-03  0.0585ms
[judge] Done. 5 traces evaluated.
2026-05-16 13:58:24.272 Submission completed
Submit is completed
Message:
  judge successfully finished
Score 83.19 max.100
Judgement Message:
  Performance: 0.06ms -> 83.19/100: 5/5 correct, avg latency 0.0585ms
```

2. 源码

见附录

3. 思考题

1. Triton 和 CUDA C 的区别是什么？

- **编程语言与抽象层级：** CUDA C 是在 C/C++ 基础上的底层扩展，需要开发者显式管理 Thread、Block、Shared Memory 以及全局同步。Triton 则是基于 Python 的 DSL，由编译器自动处理内存分配、合并访问和同步等底层细节。
- **编程模型：** CUDA 采用 SIMT 模型，开发者关注的是单个线程的行为；而 Triton 采用 Block-oriented 的编程模型，开发者直接对整个数据块进行运算描述。
- **应用场景与开发效率：** CUDA 更加通用，适合编写绝大多数 GPU 程序且能进行极限压榨；Triton 专为深度学习计算（如矩阵乘法、归一化操作）优化，能以更高的开发效率写出接近 CUDA 性能的算子代码。

2. Torch 也会自动调用 GPU 高性能库进行计算，对比 Torch 实现，你的实现是如何获得加速的？

- **Kernel Fusion：** 原生的 Torch 实现会将 RMSNorm 的计算分解为多个独立的算子（如 `pow`、`mean`、`add`、`rsqrt`、`mul`），每次执行算子都需要发起 Kernel Launch，并将中间结

果写回乃至重新读出 HBM，这在往往受限于 Memory Bound 的归一化操作中是非常大的性能瓶颈。

- 我的 Triton 实现将针对整行的操作**融合到了单一算子**中。数据仅需一次从 HBM 读入到高速互连的 SRAM（寄存器/共享内存）中，在片上完成所有的平方、求和、缩放计算后，再统一将结果写回 HBM，从而极大地节省了显存读写带宽和 Kernel 启动开销。

3. 你是如何实现 RMSNorm 中的 $\sum x_i^2$ 的？假如使用了 triton 内置函数，triton 是如何实现的？

- **我的实现：** 直接使用了 Triton 提供的内置操作 `tl.sum(x * x, axis=0)` 实现整行的平方求和。
- **Triton 的底层实现逻辑：** 编译器会将 `tl.sum` 降级并在底层生成高效的并行规约指令。
 - a. 首先通过硬件的**洗牌指令**（如 `__shfl_down_sync`）在寄存器级别进行高效的线程束内聚合。
 - b. 如果 Block 内包含多个 Warp，Triton 会分配并利用**Shared memory**，将各个 Warp 计算出的局部和写入其中，然后再依靠少数线程进行最后的 Block 级别汇总。这种做法有效掩盖了延迟并避免了低效的 Atomic Add。